

Lucie

Technical Audit & Risk Assessment

Prepared for	Lucent Brokerage (Joseph Kim, Mortgage Broker)
Prepared by	Arman Kiai, Ken Knight, Andy Chau, Jaime A. Garcia, Cody Le
Fellowship	Titan Applied AI and Entrepreneurship Summer Fellowship
Version	1.0
Date	August 18, 2026

Every figure in this report was measured live against the production API and codebase on the date above; nothing is estimated unless explicitly labeled as such.

Table of Contents

Table of Contents	2
Executive Summary	3
System Overview	4
1. Testing Methodology & Performance Evaluation	5
2. Technical Limitations & Edge Cases	8
3. Bias, Fairness, Privacy, and Misuse (Risk Assessment)	10

Executive Summary

This report evaluates Lucie, an AI social-media drafting assistant built for Lucent Brokerage (Joseph Kim, Mortgage Broker), across three dimensions requested for presentation: testing methodology and measured performance, technical limitations and edge-case behavior, and a bias/fairness/privacy/misuse risk assessment with documented mitigations.

Every quantitative figure in this report — token counts, cache hit rates, latency, and cost — was captured from live requests against the production API and codebase on the date of this report, not estimated. Every qualitative claim about system behavior (routing, compliance interception, footer enforcement) was reproduced with a real request and its real output, documented inline.

Headline findings

- Lucie runs on Claude Sonnet 5 with a ~6,987-token shared compliance/brand system prompt, cached with a 1-hour lifetime. A realistic 10-request session costs ~39% less with caching enabled than the same session would without it (§1.3).
- Prompt-level compliance instructions are not sufficient on their own: a live test today produced a draft missing its mandatory footer, caught only by a deterministic, code-level check — confirming the two-layer defense design is necessary, not redundant (§3.4).
- A second live test showed the model proactively declining to generate fair-housing-sensitive framing ("starter home for a growing family"), asking the user to reframe before any content was produced — the prompt-level layer working as intended (§2.2, §3.1).
- Campaign generation does not reliably hit its own stated post-count target: a live 3-platform/2-week request undercounted on every platform simultaneously (25–33% short). A corrective retry existed but only partially closed the gap while doubling output cost; it was removed today in favor of accepting a slightly short calendar over a guaranteed cost doubling (§2.1, §3.4).
- The application has no authentication and stores all chat/calendar/brand data exclusively in browser local storage — a real, currently policy-only (not technically enforced) privacy gap, documented in full in §3.2.

System Overview

Lucie is a Flask (Python) backend calling the Anthropic Messages API, paired with a React/Vite frontend. The backend has no database of record for user data — chat history, the content calendar, and brand/persona settings are persisted entirely in the browser's local storage. A SQLite table (chat_history) exists server-side, but only to support a content-deduplication feature (recent-idea anti-repeat context); it is not a system of record and has no relationship to user sessions or accounts, because none exist.

Property	Value (verified)
AI model	claude-sonnet-5 (confirmed via the project's .env — not the code's own fallback default, which is a cheaper model; this distinction caused a real error in an earlier draft of this project's own token audit, corrected before this report)
Backend	Flask, single Python process, no authentication, no per-user session state
Frontend	React + Vite, single-page app
Data persistence	Browser localStorage only (chats, calendar, brand/persona settings); no server-side database of record
Deployment	Vercel (frontend) + Render (backend)
Configured personas	One ("The Business Owner" — self-employed/investor, Non-QM borrower profile)

1. Testing Methodology & Performance Evaluation

1.1 Test Data & Inputs

Testing used real requests against the live application and, for the baseline comparison, the raw Anthropic API directly. Input categories exercised:

Input category	Example used	What it exercises
Ambiguous single-post request	“Write me a caption about DSCR loans” (no platform/format stated)	Clarifying-question routing (NON_CAMPAIGN_FORMAT_PROMPT shape 1)
Fully-specified single-post request	Platform + format + angle stated up front	Direct generation without a clarification round
Campaign / content-calendar request	“Build me a 1-week LinkedIn and Instagram content calendar”	CAMPAIGN_INTENT_PATTERN routing, mandatory platform/cadence clarification
Broad, time-bound (non-campaign) request	“Build me a 2 week content calendar” without platforms/cadence named	AMBIGUOUS_SCOPE_PATTERN handling — forces multi-select clarifying questions
Mutually-exclusive format edge case	Format clarifying question (Text-only / Single image / Carousel / Short video, single-select)	Single-select vs. multi-select question-type logic
Multi-platform, multi-week campaign	3 platforms × 2 weeks (up to 22 scheduled posts in one structured response)	Structured-output schema reliability under load; post-count accuracy
Fair-housing-adjacent framing	“starter home perfect for a growing family”	Bias/steering-language interception (both prompt-level and deterministic layers)
Downstream generation from existing content	Social graphic and video-script generation seeded from a previously generated post	Cross-call consistency; structured-output schema correctness
Repeat request within the cache window	Same system-prompt combination sent twice, minutes apart	Cache hit/miss behavior
Zero-shot / no-system-prompt control	Same underlying ask, sent directly to the model with no system prompt and no caching	Baseline comparison (\$1.3)

1.2 Outputs & Metrics

The application had no visibility into its own token/cache economics prior to this audit. As part of this work, a logging function (log_usage) was added to every one of the application's five Anthropic call sites, printing real input/cache-write/cache-read/output token counts from the API's own usage object on every request — captured in Render's log stream in production. Metrics tracked:

- Token consumption — input, cached-write, cached-read, and output tokens per call, per call type (single post, campaign generation, campaign clarification, video script, social graphic).
- API latency — measured wall-clock time per request.
- Cache hit rate — whether a request's shared system-prompt block was served from cache (cache_read > 0) or had to be freshly processed/written.
- Compliance adherence — the application's own deterministic post-generation checks return a compliance_flags array on every response; a non-empty array is a directly measurable compliance-miss event, not an inference.
- Persona/brand-voice adherence — evaluated qualitatively by direct comparison against the raw baseline output (tone, emoji use, structure).

Call type	Input tok.	Cache write	Cache read	Output tok.	Latency
Single post (cache warm)	1,097	0	6,987	934	~11.0s
Campaign clarification-check (cache warm)	992	0	6,030	547	—
Full campaign, 3 platforms × 2 weeks (cache warm)	1,217	0	7,795	5,277	~52s
Social graphic generation	394	4,873	0	107	—
Video script generation	376	6,369	0	809	—

Note: Social-graphic and video-script generation never hit cache in testing — their system prompts use a different block ordering than every other call site (one omits PLATFORM_PROMPT entirely), so they never share a cache entry with anything else or with each other. This is a known, documented architectural gap, not a measurement artifact.

1.3 Baseline Comparison

Two different baselines were measured, because the brief's own suggested comparisons ("raw API calls without caching" and "zero-shot generic LLM output") isolate two different things and should not be conflated.

Baseline A — zero-shot, no system prompt (quality/compliance baseline)

The same underlying request ("Write me a LinkedIn caption about DSCR loans for Joseph Kim, NMLS #684894, DRE #02190229") was sent directly to claude-sonnet-5 with no system prompt, no caching, and no structured output — approximating what a broker would get typing the same ask directly into a generic AI chat

tool.

	Baseline (zero-shot)	Lucie (optimized)
Input tokens	62	1,097 (+6,987 cached)
Output tokens	628	934
Latency	7.6s	11.0s
Emojis on LinkedIn (brand rule: none)	6 present	0 present
Mandatory footer	Present but incomplete — missing “Equal Housing Opportunity” entirely	Present, exact, verbatim
Tone	Casual, exclamatory (“Have You Heard About DSCR Loans?”)	Professional, matter-of-fact, matches configured brand voice
Output structure	Single unstructured text blob	Structured JSON (hook / body / cta / creative direction / quick details / hashtags) ready for direct UI rendering

Note: This baseline is not primarily a cost comparison — a single zero-shot call is cheaper in raw tokens than a single optimized call, because the optimized call produces substantially more structured content. The value demonstrated here is compliance and brand-fit: the zero-shot output is not publishable as-is (missing required disclosure, banned emoji use, off-brand tone), while the optimized output required no correction.

Baseline B — same system prompt, caching on vs. off (cost baseline)

This isolates the actual value of the caching work performed in this audit: the same ~6,987-token shared system prompt, sent either fresh every call (no cache_control) or once per hour with subsequent calls reading from cache. Modeled over a realistic 10-request session (Joseph drafting several posts in one sitting), using claude-sonnet-5 standard pricing (\$3/\$15 per million input/output tokens) and the real per-call token counts measured above:

	Without caching	With caching (1h TTL)
Call 1	\$0.038	\$0.059 (pays the cache-write premium)
Calls 2–10 (each)	\$0.038	\$0.019
Total for 10 calls	\$0.383	\$0.234

Note: ~39% lower cost over a realistic session. The first call in any given hour costs more with caching on than off (writing to cache costs 2× the standard input price under a 1-hour TTL), so caching only pays off once a session involves more than roughly 2–3 calls within that window — true of essentially every real session with this tool.

2. Technical Limitations & Edge Cases

2.1 Known Limitations

- No conversation memory: the backend sends only the current message to the model on every `/api/chat` call, plus a small anti-repeat “recent ideas” snippet (capped at 25 lines, ~140 characters each). There is no accumulating chat history — Lucie cannot reference anything from earlier in a conversation beyond one “Original request / Answers” round used for clarifying-question follow-ups. This also means the commonly-feared “context window fills up over a long chat” failure mode does not apply here; the actual limitation is the opposite one (no persistent memory at all).
- Entirely client-side data persistence: chat history, the content calendar, and brand/persona settings live only in the browser's `localStorage` on the device used. There is no server-side database of record, no backup, and no cross-device sync. Clearing browser data or switching devices permanently and unrecoverably loses this data.
- No authentication: the application is single-tenant with no login system. Anyone with the URL has full read/write access to whatever is stored in that browser.
- Single configured persona: the architecture supports multiple buyer personas, but only one (“The Business Owner”) is currently seeded.
- Regex-based intent routing: whether a message is treated as a single post, a full campaign, or an ambiguous/broad request is decided by two hardcoded regular expressions (`CAMPAIGN_INTENT_PATTERN`, `AMBIGUOUS_SCOPE_PATTERN`) matched against the raw message text — a pattern-matching classifier, not a model-based one. Phrasings outside the patterns' coverage could route incorrectly.
- Prompt-only compliance is not fully reliable: live-verified today (\$3.4) — the model omitted the mandatory footer on one real generation, relying on the deterministic backstop to catch it.
- Campaign post-count accuracy: the model does not reliably generate exactly as many posts per platform as it itself commits to in the same response. Live-verified today: a 3-platform/2-week request came in short on all three platforms (25–33%). A corrective-retry mechanism existed but only partially closed the gap while doubling output cost for that request; it was removed as of today's change in favor of accepting an occasionally-short calendar (logged, not silently dropped) over a guaranteed cost doubling that wasn't a guaranteed fix.
- Frontend-side randomization: loading-tip rotation and prompt-shuffle ordering use client-side `Math.random()` (Fisher-Yates shuffle) — not cryptographically significant, but a genuine reliance on the frontend rather than the server for these behaviors.

2.2 Edge Cases

- Underspecified single-post requests (no topic, platform, or format stated) are always routed to a clarifying-questions response rather than the model guessing — verified live: “give me a post idea”-style requests never silently produce a default post.
- Broad, time-bound requests (“this week,” “this month,” “strategy”) that are not full campaign requests trigger `AMBIGUOUS_SCOPE_PATTERN`, which forces every clarifying question — including platform and format

— to multi-select rather than single-select, since the request hasn't committed to one deliverable the way a specific single-post ask has.

- Mutually-exclusive choices are enforced structurally: a single post's format question (Text-only / Single image / Multi-image carousel / Short video) is always single-select, since one post cannot be more than one format at once — verified in the actual clarifying-question payload returned live.
- Follow-up detection is a string-prefix heuristic: a message beginning with “Original request:” followed by “Answers:” is treated as continuing a prior clarifying round, not as stateful session tracking (ties directly to the no-conversation-memory limitation above — this is the only continuity mechanism that exists).
- Fair-housing-sensitive framing was intercepted before generation: a live request for a post about a “starter home perfect for a growing family” did not produce content at all — the model responded with clarifying questions asking the user to reframe around property/investment fundamentals instead, exercising the prompt-level mitigation layer directly (full detail in §3.1).

3. Bias, Fairness, Privacy, and Misuse (Risk Assessment)

3.1 Bias & Fairness

The primary bias risk in mortgage-adjacent marketing content is fair-housing steering language — copy that states or implies a preference, limitation, or exclusion based on a protected characteristic (race, color, religion, sex, national origin, familial status, disability under federal law; California additionally recognizes sexual orientation, marital status, source of income, immigration/citizenship status, and age).

- Mitigation layer 1 (prompt-level): the system prompt explicitly enumerates prohibited framings (e.g. “great for young professionals,” “perfect starter home for a growing family,” “no Section 8,” “safe neighborhood”) and requires a silent self-check before presenting any draft.
- Mitigation layer 2 (deterministic backstop): a hardcoded list of ~13 known steering phrases is checked via direct string matching against every generated response, regardless of what the prompt produced. A match appends a visible, human-readable compliance-review warning to the output rather than silently discarding it.
- Both layers were exercised live in this audit, not just read from source. The prompt-level layer intercepted a steering-adjacent request before any content was generated (§2.2). The deterministic layer was separately confirmed active on a real generation that had an unrelated issue (missing footer, §3.4) — the mechanism fires on real output, not only in the abstract.

Secondary risk — tone/brand mismatch: the configured brand voice is explicitly professional, authoritative, and matter-of-fact. The zero-shot baseline in §1.3 demonstrates what ungoverned output looks like by direct contrast (casual tone, banned emoji use); the application's actual output avoided both.

Secondary risk — persona narrowness: only one buyer persona is currently configured (self-employed/investor, Non-QM). This limits content variety across borrower types as a matter of current configuration scope, not as a bias introduced by the generation process itself — the architecture supports additional personas without code changes.

3.2 Privacy Risks

This is the least-mitigated risk category in the application as it exists today, and is reported as such rather than understated.

- The chat input is a single freeform text field with no client-side or server-side filtering for sensitive input of any kind. If a broker types a real client's SSN, account number, or specific financial details, that text is sent as-is to Anthropic's API as part of the request payload.
- That same text is also written, unencrypted, into the application's local SQLite table (used only for anti-repeat idea tracking) and into the browser's localStorage — neither of which is access-controlled or designed as a durable, secure datastore.
- No authentication exists anywhere in the application, so there is no access control on any of the above — anyone with the URL and access to that specific browser can see everything stored in it.

- Current mitigation is policy-only, not technical: the operator-facing user manual explicitly instructs “never type a real client’s personal information into the chat,” but nothing in the code enforces, warns against, or redacts this. This is the single largest gap between the application’s actual risk profile and a privacy-safe design.

3.3 Potential Misuse

- Non-compliant financial advice: explicit rules forbid the model from stating or fabricating specific interest rates, APRs, fees, closing costs, or approval odds under any circumstances, and forbid guaranteed-approval or guaranteed-rate language.
- RESPA-adjacent misuse: co-marketing or “partner spotlight” content implying a referral-fee relationship with a named provider is blocked unless the user confirms a lawful marketing services agreement is in place, and even then the content itself may never describe or quantify compensation. “Refer a friend” content is restricted from being framed as compensation for referring other consumer business.
- Absolutist/hype marketing language is blocked outright via a dedicated forbidden-word list (guaranteed, easy money, lowest rates, best lender, no-brainer, stress-free, magic, amazing, game changer, rock star, crush it) — both a brand-voice control and a truth-in-advertising control.
- Pricing-offer misuse has the strictest backstop in the codebase: a dedicated phrase list (“specific rate to feature,” “promo rate,” “discounted rate,” “teaser rate,” “our rate,” “Joseph’s rate,” etc.) causes the entire draft to be withheld and replaced with a fallback message if matched — the only backstop in the application that discards output entirely rather than appending a warning, reflecting the elevated regulatory risk of pricing-specific claims.
- System-level misuse surface is low: the application has no tool use, no autonomous actions, and no integration with any social platform’s posting API. The realistic misuse surface is content-generation misuse (attempting to coax non-compliant copy out of the model), not system compromise — and the one live attempt to do so in this audit (§2.2) was declined rather than fulfilled.

3.4 Mitigation Strategies — Summary

The application’s overall compliance design is a deliberate two-layer defense: persuadable, context-aware prompt instructions, backed by blunt, deterministic, code-level checks that run regardless of what the model actually produced. Both layers were confirmed live and functioning during this audit, not merely present in source:

Layer	Mechanism	Live-verified this audit
Prompt-level	COMPLIANCE_PROMPT self-check instructions run before every draft is presented	Yes — intercepted a fair-housing-sensitive request before generation (§2.2)
Deterministic backstop	String-matched phrase/word lists (FLAGGED_STEERING_PHRASES, FORBIDDEN_WORDS, RESTRICTED_PRICING_OFFER_PHRASES) applied to every generated response regardless of prompt compliance	Yes — corrected a real footer omission on a live generation today

Layer	Mechanism	Live-verified this audit
Mandatory footer enforcement	build_footer() / enforce_compliance() guarantee the exact required NMLS/DRE/Equal Housing line is present, auto-appending it if missing	Yes — same live test as above
Structured output schemas	JSON Schema (output_config) constrains campaign and video-brief generation to a fixed shape	Yes, by design — not used on the single-post/clarification path, a known asymmetry
UI-level human-in-the-loop	No code path publishes content automatically; every draft is a review-and-edit card requiring a manual copy/download action; no social-platform posting integration exists at all	Structural — verified by inspection of every frontend action available on a generated post

Note: The publishing step is entirely manual and off-platform by construction, not by policy — there is no code path in this application that could publish content without a human explicitly copying or downloading it first. This is the single strongest structural mitigation the application has, and it is unconditional.